

lab 4

1. Create a multiply function that accepts two numbers and returns their product.

```
postgres=# CREATE FUNCTION Product_of_N(n1 integer, n2 integer) RETURNS integer AS $$ S
SELECT n1 * n2; $$ LANGUAGE SQL;
CREATE FUNCTION
postgres=# select product_of_N(5,6);
 product_of_n
-----
          30
(1 row)

postgres=#
```

=====

2. Create a hello_world function that takes a name as input and returns a personalized welcome message for that name.

```
postgres=# CREATE FUNCTION welcome(username varchar(50)) RETURNS varchar(50) AS $$ SELE
CT concat('Welcome ',username); $$ LANGUAGE SQL;
CREATE FUNCTION
postgres=# select welcome('mohamed');
  welcome
-----
Welcome mohamed
(1 row)

postgres=#
```

3. Create a function that accepts a number and determines whether it is odd or even.

```
postgres=# create function even_odd(num int)
returns text
as $$
begin
if num % 2 = 0 then
return 'even';
else
return 'odd';
end if;
end;
$$ language plpgsql;
CREATE FUNCTION
postgres=# select even_odd(5);
 even_odd
-----
      odd
(1 row)

postgres=# select even_odd(4);
 even_odd
-----
      even
(1 row)

postgres=#
```

4. Create a function that takes a Student ID as input and retrieves all information related to that student.

```
postgres=# create function search_student(stu_id int)
returns table(student_id int,name varchar(100), email varchar(100), address text, birth_date date, gender varchar(6), track_id int)
as $$
begin
return query select * from student where id = stu_id;
end;
$$ language plpgsql;
CREATE FUNCTION
postgres=# select search_student(1);
               search_student
-----
(1,Mohamed,mohamed@iti.com,Assiut,2001-04-11,Male,1)
(1 row)

postgres=# select search_student(2);
               search_student
-----
(2,Ahmed,ahmed@iti.com,Assiut,1998-01-27,Male,2)
(1 row)
```

5. Implement a function that takes the name of a subject and calculates the average grades for that subject.

```
postgres=# CREATE OR REPLACE FUNCTION avg_sub(subj_name VARCHAR)
RETURNS NUMERIC AS $$
DECLARE
    avg_grade NUMERIC;
BEGIN
    SELECT AVG(g.grade)
    INTO avg_grade
    FROM grades g
    JOIN subject s ON g.sub_id = s.id
    WHERE s.sub_name = subj_name;

    RETURN avg_grade;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# SELECT avg_sub('database');
      avg_sub
-----
83.000000000000000000
(1 row)
```

7. Create a trigger to monitor changes made to the student table, including additions, updates, and deletions. This trigger will record the time of each action and provide a description of the action in another table.

```
postgres=# CREATE TABLE stu_log (
postgres(# log_id SERIAL PRIMARY KEY,
postgres(# action_type VARCHAR(10),
postgres(# description TEXT);
CREATE TABLE
```

```

postgres=# CREATE FUNCTION log_stu()
postgres=# RETURNS TRIGGER AS $$
postgres$# BEGIN
postgres$#     IF TG_OP = 'INSERT' THEN
postgres$#         INSERT INTO stu_log(action_type, description)
postgres$#         VALUES ('INSERT', 'New student added: ' || NEW.e_name);
postgres$#
postgres$#     ELIF TG_OP = 'UPDATE' THEN
postgres$#         INSERT INTO stu_log(action_type, description)
postgres$#         VALUES ('UPDATE', 'Student updated: ' || NEW.e_name);
postgres$#
postgres$#     ELIF TG_OP = 'DELETE' THEN
postgres$#         INSERT INTO stu_log(action_type, description)
postgres$#         VALUES ('DELETE', 'Student deleted: ' || OLD.e_name);
postgres$#     END IF;
postgres$#
postgres$#     RETURN NULL;
postgres$# END;
postgres$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# CREATE TRIGGER trg_stulogs
postgres=# AFTER INSERT OR UPDATE OR DELETE ON student
postgres=# FOR EACH ROW
postgres=# EXECUTE FUNCTION log_stu();
CREATE TRIGGER
postgres=# INSERT INTO student (e_name, email, address, birth_date, gender, track_id)
VALUES ('Ali', 'ali@iti.com', 'Assiut', '2006-08-11', 'Male', 2);
INSERT 0 1
postgres=# select * from stu_log;
 log_id | action_type |      description
-----+-----+-----
      1 | INSERT     | New student added: Ali
(1 row)

postgres=# █

```